



Практическая работа: "Управление рестораном"

Представьте себе, что вы – владелец уютного ресторана «Пицца-суши», который славится своими уникальными блюдами и атмосферой. Каждый день к вам приходят голодные гости, готовые насладиться кулинарными шедеврами вашего шеф-повара. Однако, как и в любом успешном ресторане, у вас возникла проблема: хаос с заказами!

Ваши официанты записывают заказы на салфетках (да-да, вы не ослышались!), а повара иногда забывают, что именно нужно готовить. Ваша кухня превращается в поле битвы, где каждый пытается выжить среди кипящих кастрюль и летающих тарелок. В том числе у Вас нет понимания какие гости к Вам ходят чаще всего, чтобы выстроить систему бонусов. А также Вы не понимаете, как оценить работоспособность официантов и кому давать премию в конце месяца, потому что не ведете счет выполненным заказам. Вы понимаете, что пора навести порядок.

Чтобы справиться с этой ситуацией, вы решили разработать систему учета заказов и блюд с использованием LINQ в C#. Эта система должна помочь вам отслеживать заказы, блюда и клиентов, а также предоставить отчеты о самых популярных блюдах и клиентах, о самых трудолюбивых официантах (которые выполняют наибольшее число заказов). Интерфейс должен быть представлен в виде консольного приложения с удобным меню по выбору функционала. **Программа должна быть интерактивной.**

Техническое задание

Итак, весь процесс, который Вам предстоит автоматизировать, осуществляется следующим образом. Клиенты приходят в Ваш ресторан и садятся за столик, выбирают из меню определенные позиции и приглашают официанта. Официант должен запомнить следующие: номер столика, id клиента (если он пользуется программой лояльности), все позиции заказа и в дальнейшем сформировать заказ (не забыв конечно же указать свой id официанта). Заказ передается в дальнейшем на кухню.

В Вашем ресторане 2 кухни: японская и итальянская. В каждой кухне работает по 3 повара, всего 6. При поступлении заказа на кухню, все позиции заказа распределяются между поварами по следующему правилу: у каждого повара не может быть более 3 блюд в изготовлении одновременно, и, конечно же, каждый повар может готовить только то блюдо, которое он умеет (блюда итальянской или японской кухни).

Описание классов

1. Блюдо (`class Dish`)

Свойства:

- `int Id` – уникальный идентификатор блюда,
- `string Name` – название блюда,
- `CuisineType Cuisine` – категория блюда // перечисление японское или итальянское
- `int Price` – стоимость блюда

2. Клиент (`class Customer`)

Свойства:

- `int Id` – уникальный идентификатор клиента,
- `string Name` – имя клиента.

3. Заказ (`class Order`)

Свойства:

- `Id` – уникальный идентификатор заказа,
- `int TableNumber` – номер столика,
- `int CustomerId` – идентификатор клиента,
- `int WaiterId` – идентификатор официанта,
- `List<Dish> Dishes` – коллекция блюд,
- `decimal TotalPrice` – общая стоимость заказа.

Методы:

`void AddDish(Dish id)` – метод, позволяющий добавить новое блюдо в заказ.

4. Шеф (`class Chef`)

Свойства:

- `string Name` - Имя
- `int OrdersHandled` – количество текущих заказов
- `CuisineType TypeOfCuisine` – тип кухни (японская или итальянская)

5. Официант (`class Waiter`)

Свойства:

- `int Id` – уникальный идентификатор
- `string Name` - имя
- `int OrdersHandled` – количество принятых заказов.

5. Ресторан (`class Restaurant`)

Свойства:

- `List<Chef> Chefs`
- `List<Order> Orders`
- `List<Dish> Dishes`
- `List<Waiter> Waiters`
- `List<Customer> Customers`

Методы:

1. `Dish FindOfDish(int id)` – поиск блюда по идентификатору.
2. `TakeOrder(Order order)` – сформировать заказ. В рамках этого метода необходимо разделить все блюда на соответствующие кухни и назначить на каждое блюдо своего шеф-повара. Предусмотрите вывод информации о необходимости ожидать заказ более часа (сделать это можно разными способами), если нет свободных шефов.

Также в рамках класса `Restaurant` Вам необходимо предусмотреть возможность формировать следующие отчеты, реализовав методы:

3. `ShowPopularDishes` – отчет о самых популярных блюдах

Ваша цель здесь - подсчитать количество раз, когда каждое блюдо было заказано, и вывести все позиции в порядке убывания популярности. Для этого Вы можете использовать вариант следующего LINQ запроса: используйте оператор `SelectMany` чтобы получить плоский список всех блюд из всех заказов. После того как Вы получите все блюда в одном списке, сгруппируйте их по

имени блюда. Далее, Вам необходимо пройтись по каждой группе и создать новый анонимный объект для каждой группы, который будет иметь следующую структуру:

- `g.Key` — это имя блюда (так как мы группировали по имени).
- `g.Count()` — это количество элементов в группе, то есть количество раз, когда данное блюдо было заказано.

В результате Вы должны получить коллекцию анонимных объектов, каждый из которых содержит имя блюда и количество раз, когда оно было заказано. Последним шагом отсортируйте полученные анонимные объекты по количеству заказов в порядке убывания, используя оператор `OrderByDescending`.

4. `ShowHardworkingWaits()` – отчет о работе официантов (официант – количество выполненных заказов).

В данном случае, Вам необходимо сформировать коллекцию, содержащую официантов, отсортированных по количеству обработанных заказов. Для этого Вы можете использовать метод `OrderByDescending`. В качестве параметра передайте лямбда-выражение, которое будет указывать, что сортировка должна происходить по свойству `OrdersHandled` каждого объекта.

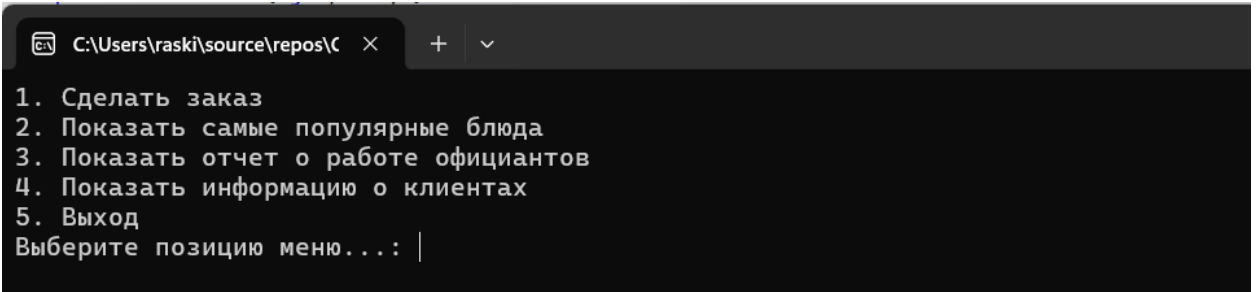
5. `ShowReportAboutCustomers()` – отчет о клиентах

Цель этого отчета - получить сводную информацию о заказах клиентов в разрезе общей суммы всех заказов и количества этих заказов, что может быть полезно для анализа продаж или расчета бонусов. Для формирования этого отчета Вы можете использовать LINQ запрос следующей структуры. Вам необходимо соединить две коллекций: `Orders` и `Customers`. Используйте для этого оператор `Join`. Вы должны соединить заказы с клиентами по полям `IdOfCustomer` из заказа и `Id` из клиента. Это позволит получить информацию о клиенте для каждого заказа. После соединения мы группируем заказы по имени клиента (`client.Name`). Используйте для этого оператор `group by`. Результат группировки сохраняется в переменной `g`. В каждой группе `g` будут находиться все заказы, сделанные конкретным клиентом. Далее сформируйте новый анонимный объект для каждой группы, который будет содержать необходимые данные.

- `ClientName`: имя клиента (ключ группы).
- `TotalDishes`: общее количество денег, потраченных клиентом на заказы.
- `TotalOrdersCount`: общее количество заказов, сделанных клиентом.

Вариант работы консольного приложения:

1. Вид меню:



```
C:\Users\raski\source\repos\C  ×  +  ▾
1. Сделать заказ
2. Показать самые популярные блюда
3. Показать отчет о работе официантов
4. Показать информацию о клиентах
5. Выход
Выберите позицию меню...: |
```

2. Осуществление заказа:

```
C:\Users\raski\source\repos\C x + v
1. Сделать заказ
2. Показать самые популярные блюда
3. Показать отчет о работе официантов
4. Показать информацию о клиентах
5. Выход
Выберите позицию меню...: 1
Введите номер столика...: 1
Введите id официанта.: 1
Введите id клиента.: 1
Введите id блюда (или наберите <готово>, чтобы закончить): 1
Введите id блюда (или наберите <готово>, чтобы закончить): 2
Введите id блюда (или наберите <готово>, чтобы закончить): 1
Введите id блюда (или наберите <готово>, чтобы закончить): 2
Введите id блюда (или наберите <готово>, чтобы закончить): 3
Введите id блюда (или наберите <готово>, чтобы закончить): готово
Позиция Паста Альфредо назначена на Chef F
Позиция Паста Карбанара назначена на Chef F
Позиция Паста Альфредо назначена на Chef F
Свободных шефов нет, ожидание более часа
Свободных шефов нет, ожидание более часа
```

3. Отчет о самых популярных блюдах:

```
1. Сделать заказ
2. Показать самые популярные блюда
3. Показать отчет о работе официантов
4. Показать информацию о клиентах
5. Выход
Выберите позицию меню...: 2
Популярные позиции меню:
Паста Альфредо: 2
Паста Карбанара: 2
Пицца пепперони: 1
```

4. Отчет о работе официантов:

```
1. Сделать заказ
2. Показать самые популярные блюда
3. Показать отчет о работе официантов
4. Показать информацию о клиентах
5. Выход
Выберите позицию меню...: 3
Официанты и количества выполненных заказов:
Waiter A: 1 заказов выполнено
Waiter B: 0 заказов выполнено
```

5. Вывод информации о клиентах:

```
1. Сделать заказ
2. Показать самые популярные блюда
3. Показать отчет о работе официантов
4. Показать информацию о клиентах
5. Выход
Выберите позицию меню...: 4
Покупатели и их заказы:
Иван: 5550 - общая сумма заказов, 1 - общее количество заказов
Мария: 3000 - общая сумма заказов, 1 - общее количество заказов
```